AI Agents

Introduction

AI Agents represent an exciting development in Generative AI, enabling Large Language Models (LLMs) to evolve from assistants into agents capable of taking actions. AI Agent frameworks enable developers to create applications that give LLMs access to tools and state management. These frameworks also enhance visibility, allowing users and developers to monitor the actions planned by LLMs, thereby improving experience management.

Learning goals

After taking this lesson, you'll be able to:

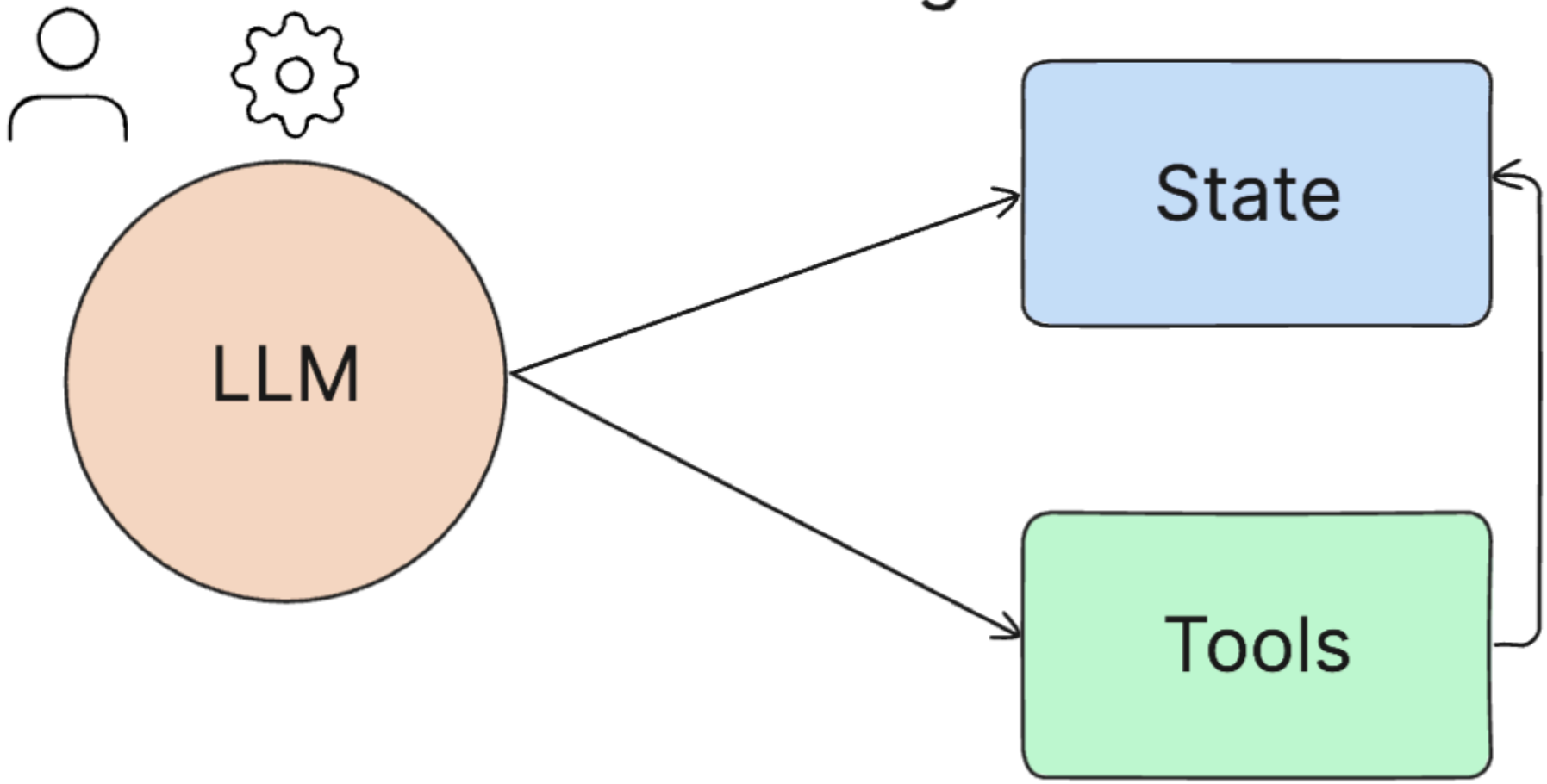
- Explain what AI Agents are and how they can be used.
- Have an understanding of the differences between some of the popular AI Agent Frameworks, and how they differ.
- Understand how AI Agents function in order to build applications with them.

What Are AI Agents?

AI Agents are a very exciting field in the world of Generative AI. With this excitement comes sometimes a confusion of terms and their application. To keep things simple and inclusive of most of the tools that refer to AI Agents, we are going to use this definition:

AI Agents allow Large Language Models (LLMs) to perform tasks by giving them access to a **state** and **tools**.

What is an Agent?



LLM

- "Choose Your Own Adventure"

Let's define these terms:

Large Language Models - These are the models referred throughout this course such as GPT-5, GPT-4o, and Llama 3.3, etc.

State - This refers to the context that the LLM is working in. The LLM uses the context of its past actions and the current context, guiding its decision-making for subsequent actions. AI Agent Frameworks allow developers to maintain this context easier.

Tools - To complete the task that the user has requested and that the LLM has planned out, the LLM needs access to tools. Some examples of tools can be a database, an API, an external application or even another LLM!

These definitions will hopefully give you a good grounding going forward as we look at how they are implemented. Let's explore a few different AI Agent frameworks:

LangChain Agents

LangChain Agents is an implementation of the definitions we provided above.

To manage the **state** , it uses a built-in function called the `AgentExecutor` . This accepts the defined `agent` and the `tools` that are available to it.

The `Agent Executor` also stores the chat history to provide the context of the chat.

LangChain Agents

```
agent_executor =
AgentExecutor(agent=agent,
tools=tools, verbose=True)
```

State
AgentExecutor
Chat History

```
agent_executor.invoke({"input":
"hi! my name is bob",
"chat_history": []})
```

LLM

Context
System Message

```
from langchain_openai import
ChatOpenAI
llm = ChatOpenAI(model="gpt-3.5-
turbo", temperature=0)
```

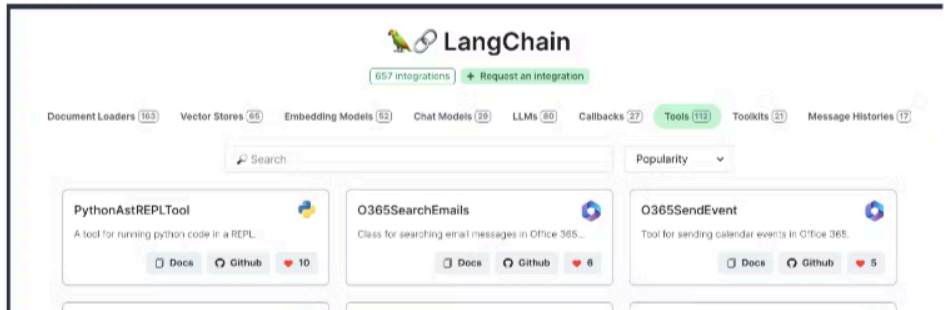
```
You Are a Helpful Assistant
```

```
agent = create_openai_functions_agent(llm,
tools, prompt)
```

Tools
Search,
Services,
Databases

```
from
langchain_community.tools.tavily_se
arch import TavilySearchResults

tools = [search, retriever_tool]
```



AutoGen

The next AI Agent framework we will discuss is AutoGen. The main focus of AutoGen is conversations. Agents are both **conversable** and **customizable**.

Conversable - LLMs can start and continue a conversation with another LLM in order to complete a task. This is done by creating `AssistantAgents` and giving them a specific system message.

```
autogen.AssistantAgent( name="Coder", llm_config=llm_config, ) pm = autogen.AssistantAgent( name="Product_manager", system_message="Creative in software product ideas.", llm_config=llm_config, )
```

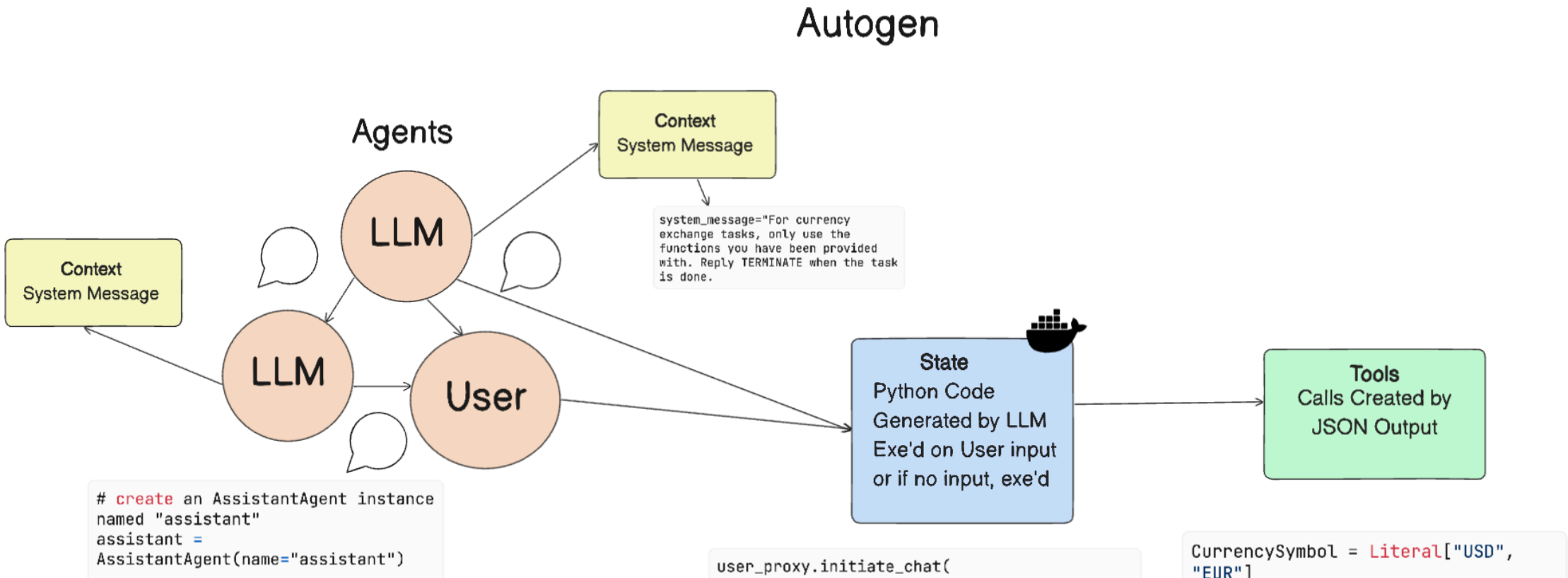
Customizable - Agents can be defined not only as LLMs but be a user or a tool. As a developer, you can define a `UserProxyAgent` which is responsible for interacting with the user for feedback in completing a task. This feedback can either continue the execution of the task or stop it.

```
user_proxy = UserProxyAgent(name="user_proxy")
```

State and Tools

To change and manage state, an assistant Agent generates Python code to complete the task.

Here is an example of the process:



Microsoft Agent Framework

Microsoft Agent Framework is Microsoft's open-source SDK for building AI Agents and multi-agent systems in both **Python** and **.NET**. It brings together the strengths of two earlier Microsoft projects — the enterprise features of **Semantic Kernel** and the multi-agent orchestration of **AutoGen** — into a single, supported framework. If you are starting a new agent project today, this is the recommended successor to AutoGen.

The framework scales from a single **chat agent** all the way up to complex **multi-agent workflows**, and it integrates directly with Microsoft Foundry, Azure OpenAI, and OpenAI. It also provides built-in observability through OpenTelemetry so you can trace exactly what your agents are doing.

State and Tools

State - The framework manages conversation context for you through **threads**. An agent keeps track of the message history (user requests, tool calls, and their results), so each turn builds on the previous ones. Threads can also be persisted, allowing a conversation to be paused and resumed later.

Tools - You give an agent tools by passing plain Python functions. Type-annotated parameters are automatically turned into a schema, so the model knows how and when to call them (function calling). The framework also supports Model Context Protocol (MCP) servers and hosted tools such as a code interpreter.

Here is an example of a single agent with a custom tool:

```python
import asyncio
from typing import Annotated

from pydantic import Field
from agent_framework import Agent
from agent_framework.openai import OpenAIChatClient


def get_weather(
    location: Annotated[str, Field(description="The location to get the weather for.")],
) -> str:
    """Get the weather for a given location."""
    return f"The weather in {location} is sunny with a high of 22°C."


async def main():
    agent = Agent(
        client=OpenAIChatClient(),
        instructions="You are a helpful assistant that can answer weather questions.",
        tools=[get_weather],
    )

    response = await agent.run("What's the weather in Amsterdam?")
    print(response)


asyncio.run(main())
```

To connect to Azure OpenAI in Microsoft Foundry instead, pass your endpoint and credentials to the client:

```
from azure.identity.aio import AzureCliCredential
from agent_framework.openai import OpenAIChatClient

client = OpenAIChatClient(
    model="my-gpt-5-mini-deployment",
    azure_endpoint="https://my-resource.openai.azure.com",
    credential=AzureCliCredential(),
)
```

Multi-agent workflows

Where the framework really stands out is orchestrating several agents together. For example, you can run agents one after another (each passing its context to the next) or fan out to several agents in parallel and aggregate their results:

```
from agent_framework.orchestrations import SequentialBuilder, ConcurrentBuilder

# Run agents in sequence, passing the conversation context along the chain
sequential = SequentialBuilder(participants=[researcher, writer, editor]).build()

# Fan out to agents in parallel, then aggregate their responses
concurrent = ConcurrentBuilder(participants=[analyst_a, analyst_b, analyst_c]).build()
```

To install the framework and get started:

```
pip install agent-framework-core
# Optional integrations
pip install agent-framework-openai        # OpenAI and Azure OpenAI
pip install agent-framework-foundry       # Microsoft Foundry
```

You can explore more in the Microsoft Agent Framework repository and the official documentation.

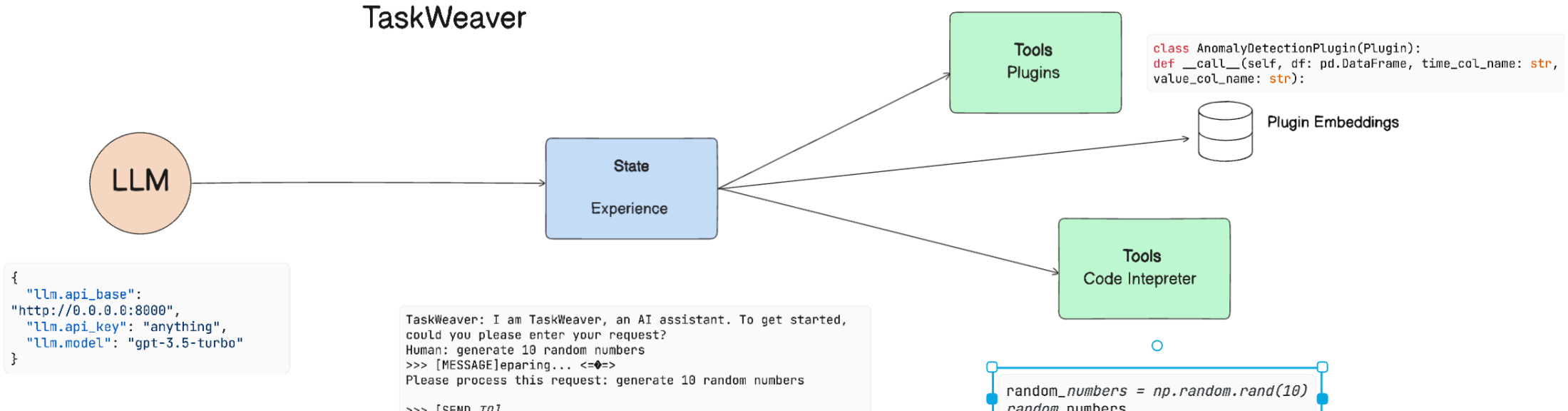
Taskweaver

The next agent framework we will explore is Taskweaver. It is known as a "code-first" agent because instead of working strictly with `strings` , it can work with DataFrames in Python. This becomes extremely useful for data analysis and generation tasks. This can be things like creating graphs and charts or generating random numbers.

State and Tools

To manage the state of the conversation, TaskWeaver uses the concept of a `Planner`. The `Planner` is a LLM that takes the request from the users and maps out the tasks that need to be completed to fulfill this request.

To complete the tasks the `Planner` is exposed to the collection of tools called `Plugins`. This can be Python classes or a general code interpreter. This plugins are stored as embeddings so that the LLM can better search for the correct plugin.



JARVIS

The last agent framework we will explore is JARVIS. What makes JARVIS unique is that it uses an LLM to manage the `state` of the conversation and the `tools` are other AI models. Each of the AI models are specialized models that perform certain tasks such as object detection, transcription or image captioning.

